

## Should Agents Reason Linearly or as a Tree?

Baseline (IO) vs. Self-Refine vs. Tree-of-Thought vs. Agentic-Program-Repair

Same models, same prompts, same tasks. Different reasoning loop.

# Methods

| Method                 | Paper               | What it does                                                                                |
|------------------------|---------------------|---------------------------------------------------------------------------------------------|
| Baseline (IO)          | —                   | One-shot autoregressive rollout                                                             |
| Self-Refine            | Madaan et al. 2023  | LLM critiques its own answer at every round                                                 |
| Tree-of-Thought        | Yao et al. 2023     | Keeps several half-finished attempts alive, scores them, expands the best, prunes the rest. |
| Agentic-Program-Repair | Maddila et al. 2025 | Make N independent attempts, keep those that pass tests.                                    |

## Summary: Agentic Program Repair

Use deterministic, verifiable tests to determine correctness. Think: Test-Driven Development.

In the paper: an agent reads the failing tests, edits code, runs the tests, and repeats. Their headline result is not a clever algorithm — it is **feedback plus retries**:

| Their system                             | Solve rate   |
|------------------------------------------|--------------|
| Agent alone                              | 28.5%        |
| + static analysis feedback               | 34.1%        |
| + <b>test execution feedback</b>         | <b>43.9%</b> |
| Best single patch of many attempts       | 46.3%        |
| <b>5 repeated runs of the whole loop</b> | <b>61.0%</b> |

The Engineering Agent is an internal Meta system and is **not open-source**, so our implementation is built from the paper alone — keeping the transferable part: fresh attempts, judged by tests.

## Our Experiments and task distribution

| Family          | Tasks | How we grade it                 | Task IDs                                                                                                              |
|-----------------|-------|---------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Game of 24      | 6     | Exact arithmetic check          | g24-0 ... g24-5                                                                                                       |
| Coding          | 5     | Real unit tests in a subprocess | prog-merge_intervals , prog-rotate , prog-kth_largest , prog-parse_formula ,<br>prog-merge_sorted_arrays              |
| Letter counting | 6     | Exact count                     | count-strawberry , count-raspberry , count-Mississippi , count-bookkeeper ,<br>count-dreadnought , count-refrigerator |
| Total           | 17    |                                 |                                                                                                                       |

**Nothing is graded by a model.** Every pass/fail comes from arithmetic, a test run, or a string compare.

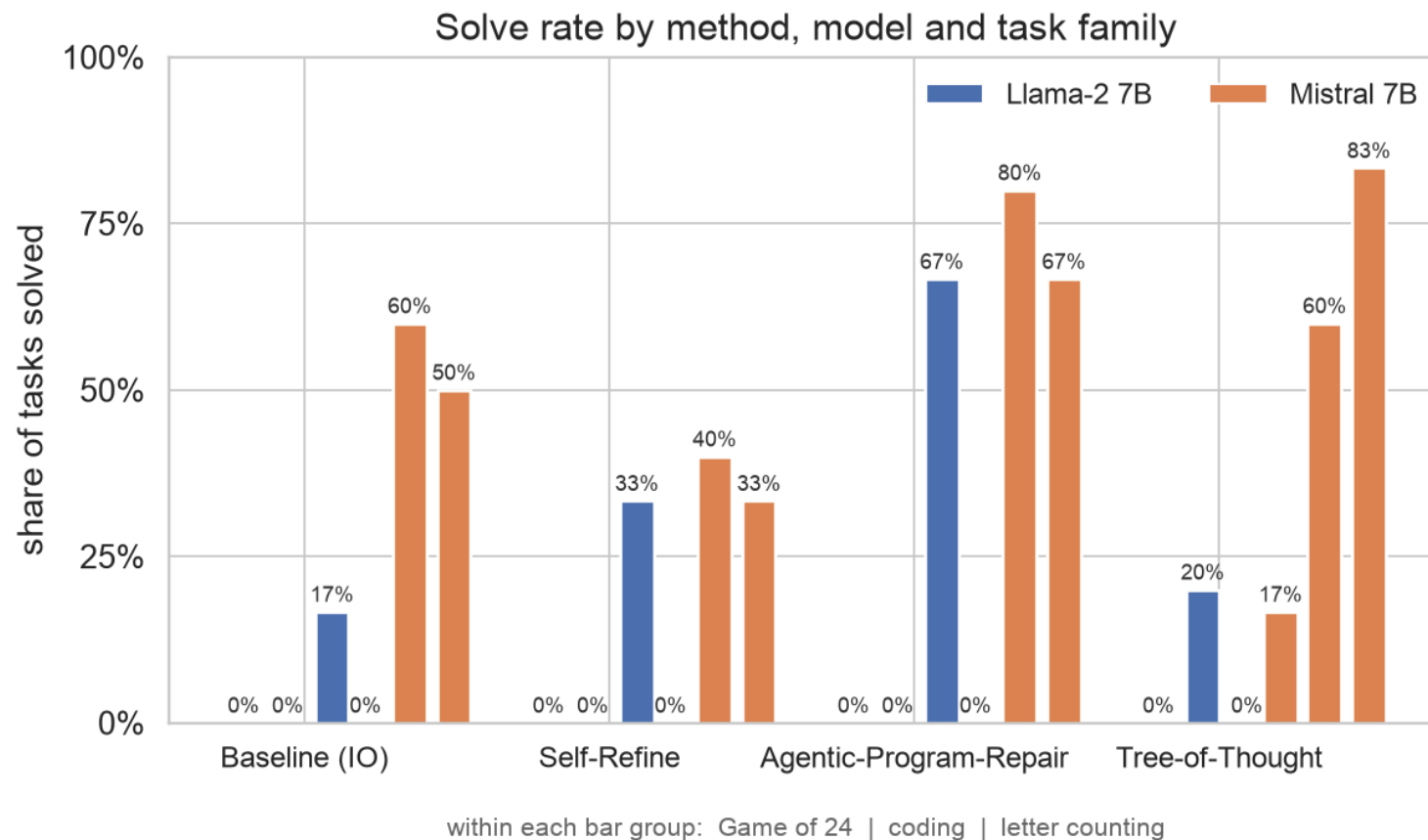
Each task gets 8 runs (2 models × 4 methods) → **136 runs**. Six were solved by nobody: g24-1 ... g24-5 and prog-parse\_formula .

# Experiment Setup

|         |                                                                                        |
|---------|----------------------------------------------------------------------------------------|
|         |                                                                                        |
| Models  | llama2:7b-chat and mistral:7b-instruct — two 7B models from the same era as the papers |
| Serving | Ollama, 2 GPUs, NUM_PARALLEL=4 , 6 tasks in flight                                     |
| Scale   | 2 models × 4 methods × 17 tasks = <b>136 runs</b>                                      |
| Budget  | Same prompts for all four methods; 60 model calls per task maximum                     |
| Metrics | Solve rate · generated tokens · model calls · wall-clock time                          |

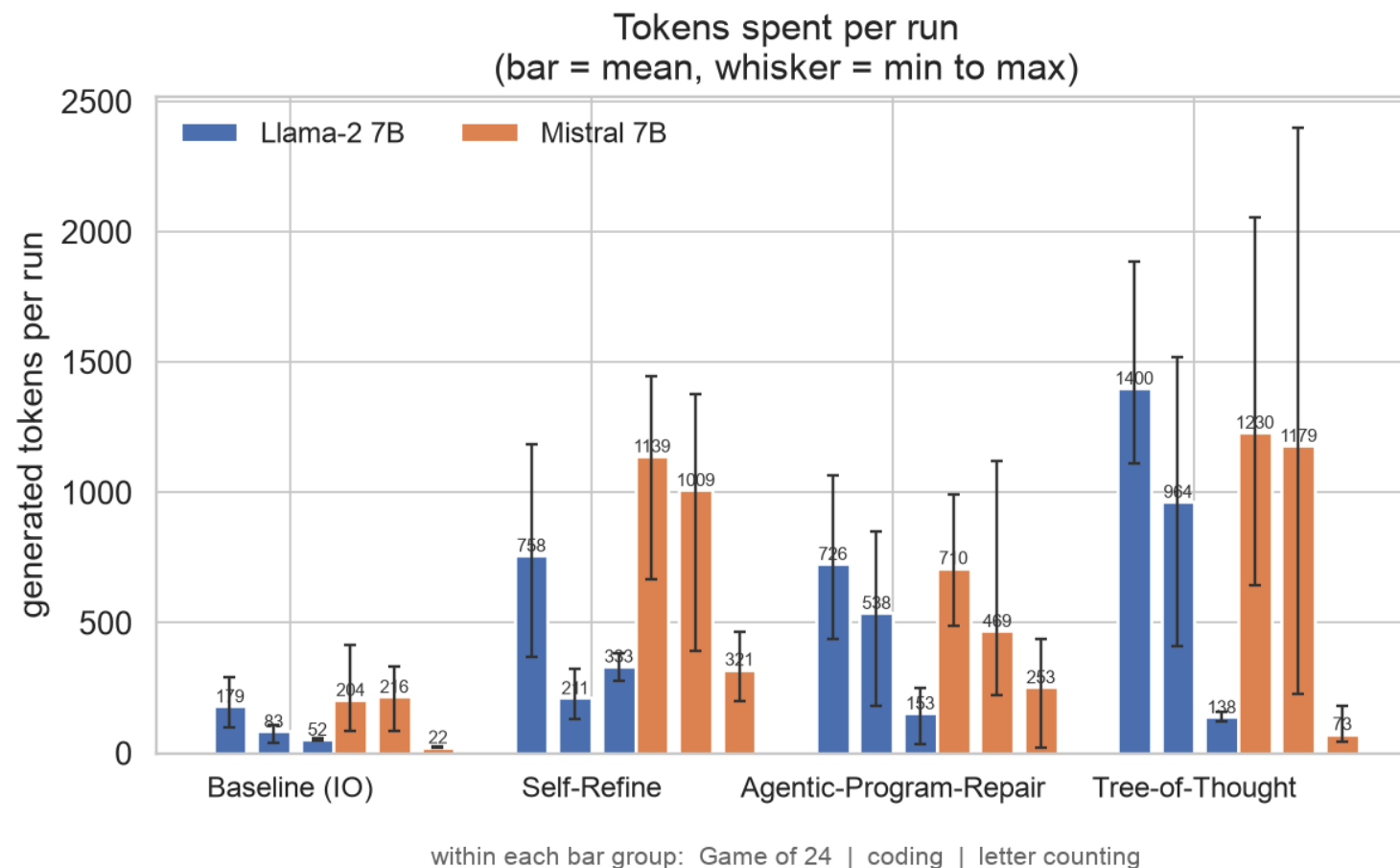
Cost is reported per *solved* task — total tokens spent divided by tasks actually solved.

## Success rate



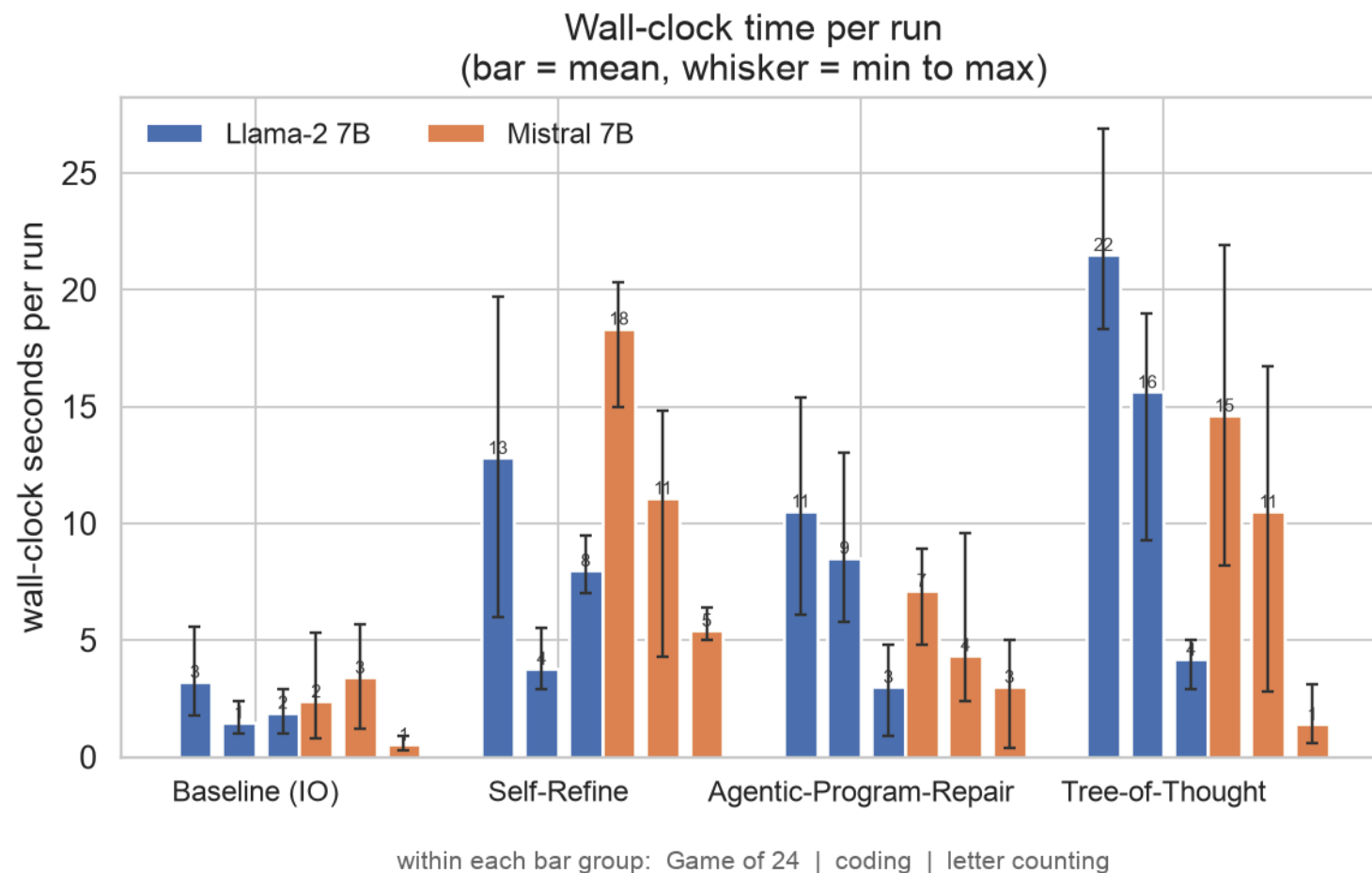
**Findings:** Agentic-PR leads overall (35%) and on both coding and counting. Tree-of-Thought only works on the stronger model (53% Mistral vs 6% Llama-2). Game of 24 is a wall: 1 solve in 48.

# Tokens spent



**Findings:** Tree-of-Thought costs 9× more per solve on Llama-2 than on Mistral (14,045 vs 1,524), same code. Agentic-PR is the cheapest route to accuracy.

## Wall-clock time



**Findings:** Baseline 2.1s per run, Agentic-PR 6.1s, Self-Refine 10.0s, Tree-of-Thought 11.2s. Time tracks how many calls a method makes — not how good it is.



## What I take away

1. **Verification is the win, not the tree.** Tests turn extra compute into correct answers. More thinking without a check mostly buys longer wrong answers.
2. **The methods are not interchangeable.** Agentic-PR owns counting and coding; Tree-of-Thought's best case is a harder task on a stronger model.
3. **Self-Refine is the weakest link here.** Most expensive, least accurate — worse than one-shot on every family.
4. **Time tracks the loop, not the result.** Baseline 2.1s per run, Agentic-PR 6.1s, Self-Refine 10.0s, Tree-of-Thought 11.2s — and the extra time does not buy accuracy for the two model-driven methods.
5. **Cost depends on the model as much as the method.** Tree-of-Thought cost 9× more per solve on Llama-2 than on Mistral, with identical code.

*Caveats: 34 runs per method is directional, not statistical. Two 7B models from 2023. Game of 24 was out of reach, so the math family contributes almost nothing. Timings include GPU queue time.*

## Appendix A — What APR is in our implementation

Independent fresh attempts, verified by tests. Not self-refine, and not a combination.

| Attempt | What the model sees                                                                       |
|---------|-------------------------------------------------------------------------------------------|
| 1       | the problem prompt, alone                                                                 |
| 2       | the problem prompt + the rejected answer + <i>"start again with a DIFFERENT approach"</i> |
| 3       | the same, with attempt 2 as the rejected answer                                           |

Each attempt is a complete, independent answer. Test results decide **whether to retry** — nothing is carried forward as a partial solution.

**Honest gap vs. the paper:** we tell the model *"the verifier rejected it"*, not *which test failed*. APR feeds the real test output back into the loop, so our signal is weaker than theirs.

# Appendix B — Key parameters (1 of 2)

| Parameter   | Value                                                    |
|-------------|----------------------------------------------------------|
| Models      | llama2:7b-chat , mistral:7b-instruct (7B, 2023-era)      |
| Serving     | Ollama 0.40.0, CUDA, 2 × RTX PRO 6000 (95 GB each)       |
| Concurrency | NUM_PARALLEL=4 , 6 runs in flight ( --jobs 6 )           |
| Tasks       | 17 per model per method (6 math / 5 coding / 6 counting) |
| Runs        | 2 models × 4 methods × 17 tasks = <b>136</b>             |
| Call cap    | 60 model calls per task (max observed: 10)               |
| Output cap  | 450 tokens per generation                                |

## Appendix B — Key parameters (2 of 2)

| Method                 | Setting                                                                                    |
|------------------------|--------------------------------------------------------------------------------------------|
| Baseline (IO)          | temperature 0.0, 1 call                                                                    |
| Self-Refine            | temperature 0.7, max 4 rounds; critique at temperature 0.0, 120 tokens                     |
| Agentic-Program-Repair | temperature 0.9, 3 independent attempts                                                    |
| Tree-of-Thought        | temperature 0.8; depth 3 (math) / 2 (others); expand 2; beam 2; scoring at temperature 0.0 |

**Verifiers** ( `tasks.py` ): unit tests in a subprocess with a 10 s timeout; an exhaustive solver for Game of 24; exact matching for counts.

**Deliberate fairness choice:** `--iters 4` (Self-Refine's paper default) and `--candidates 3` give all methods comparable numbers of attempts.